

Airline Passenger Reservation System - ver.C

综合实验报告

实验名称: 程序设计实验

题 目: 航空客运订票系统

作 者: Jelliny

2026 年 1 月 23 日

题目名称：航空客运订票系统

一、课程设计目的

1. 进一步掌握和利用 C 语言进行行程设计的能力；
2. 进一步理解和运用结构化程序设计的思想和方法；
3. 初步掌握开发一个小型实用系统的基本方法；
4. 学会调试一个较长程序的基本方法；
5. 学会利用流程图表示算法；
6. 掌握书写程序设计开发文档的能力（书写课程设计报告）。

二、题目描述

航空客运订票系统作为现代民航运营的核心业务支撑平台，承担着航线信息管理、票务交易等重要职能。本系统旨在构建一个功能完备的航空客运订票解决方案，落实航班信息管理、票务查询、订购、候补及退票服务等核心业务流程。

三、功能分析

航空客运订票系统包含以下基本功能：

1. 航班信息管理功能

航空客运订票系统管理员输入每条航线需存储的详细运营信息，数据包括：起降机场信息（起点站名、终点站名）、航班标识信息（航班号）、运营时间（飞行周日、航行时刻）、票务信息（乘员定额、当前余票量、票价）以及客户信息（已订票乘客名单、候补用户名单）。

其中，乘客名单需记录乘客姓名及所需票数等详细信息，为后续票务处理提供数据支持。

2. 航班查询功能

用户可通过输入出发地和目的地信息，获取相关航班的详细信息。查询结果包括：航班号、飞行周日、航行时刻、乘员定额、余票量、票价。

3. 票务预订功能

订票业务流程包括：根据用户输入的航班号、订票数额等需求，查询该航班余票状态。

当该航班满足用户订票需求，即余票充足时，则为用户办理订票手续并更新数据库；反之，若票额不足，则需重新询问用户是否需要调整订票需求。

首先，根据飞行周日为用户推荐同天、但不同航班号和航行时刻的航班作为替代方案，若用户同意调整航班，则引导其办理新航班订票；若用户不同意更换航班，则将其信息加入本航班候补队列等待余票。

4. 退票后补功能

退票业务流程包括：验证用户退票资格、执行退票操作并释放相应票额、自动检索该航班候补队列。

本系统将按照先进先出原则处理候补用户，若退票数量满足队列首位用户的订票数额需求，则自动完成票务转移；否则，将依次询问后续候补用户。

四、系统设计

1. 程序总体设计

航空客运订票系统运行过程：首先，系统界面外观；其次，管理员输入每条航线需存储的详细运营数据。然后，乘客依据需求选择航班查询、订票、退票业务，并根据系统指引办理相应业务。业务结束后存储最新数据并返回航空客运订票系统初始界面。

所有流程均按行输入信息，通过回车键转向输入下一行信息或进行下一步骤。

本系统主流程如图 1 所示。

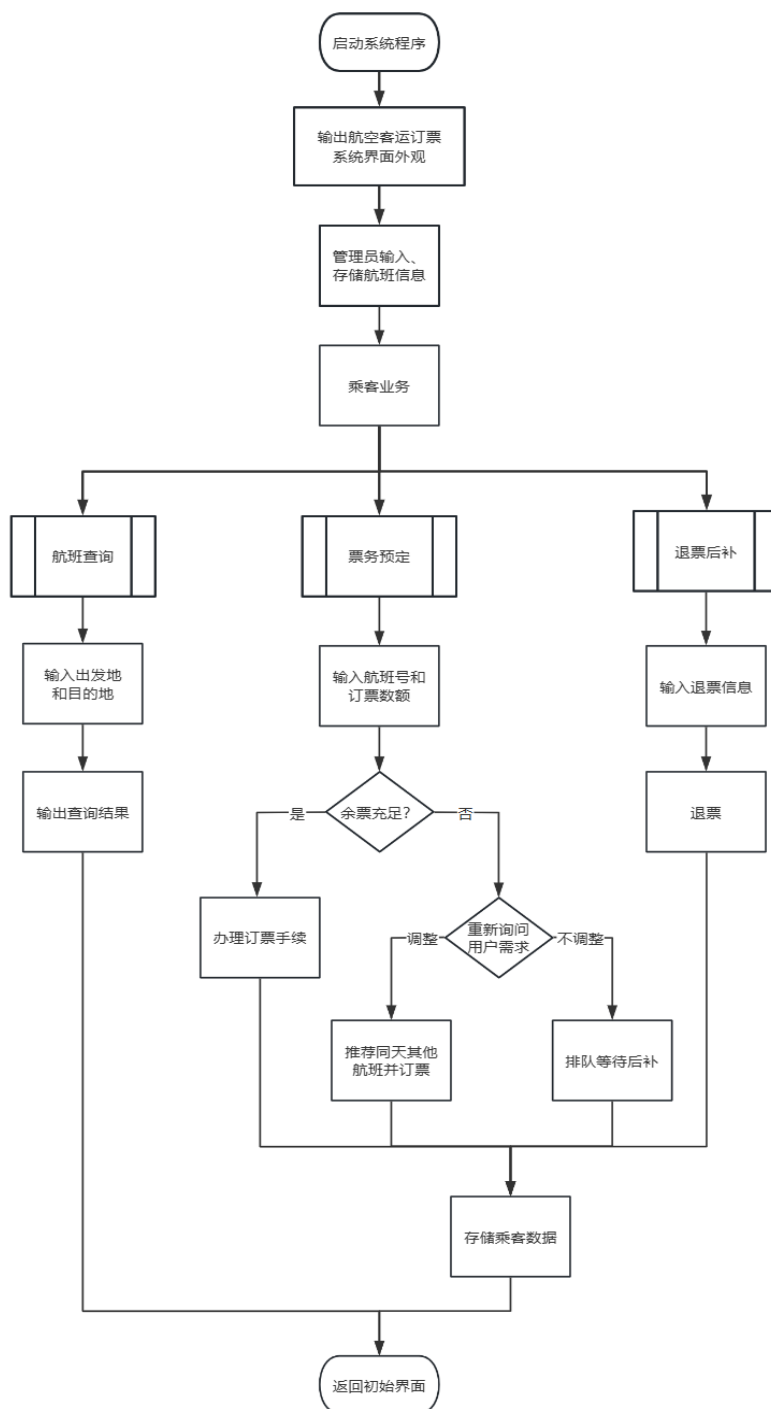


图 1 航空客运订票系统流程图

2. 界面设计

航空客运订票系统的界面如图 2、图 3 所示，具体设计如下：

(1) 网页背景采用黑色。其中，用蓝色框线分割主界面功能区，提示用户操作范围。

(2) 主页面第 1 行是界面标题“Airline Booking 航空客运订票系统”，界面标题用蓝色框线突出。

(3) 主界面标题显示框下面有 7 行业务选项，每行对应 1 个业务。业务提示包含操作人员和该业务主要内容。

(4) 主界面最下方一行是对用户选择业务的操作提示，用户在该处输入合法数字，即可进入对应业务功能界面进一步操作。

(5) 功能界面分为 4 个栏目，从上到下依次为：业务类型、业务内容、用户操作栏、系统反馈。每个栏目之间通过蓝色分割线隔开。

(6) 4 个栏目的显示内容由蓝色、白色、绿色、红色、黄色 5 种颜色的文字构成。

蓝色文字为系统提示信息，如业务类型小标题、分割线、二次确认提示等；白色文字为用户输入信息；绿色文字为操作成功提示信息，如票额充足、输入合法、操作成功等；红色文字为操作失败提示信息，如票额不足、输入不合法、操作失败等；黄色文字为交互提示信息，如推荐航班、排队候补、保存及删除数据等。

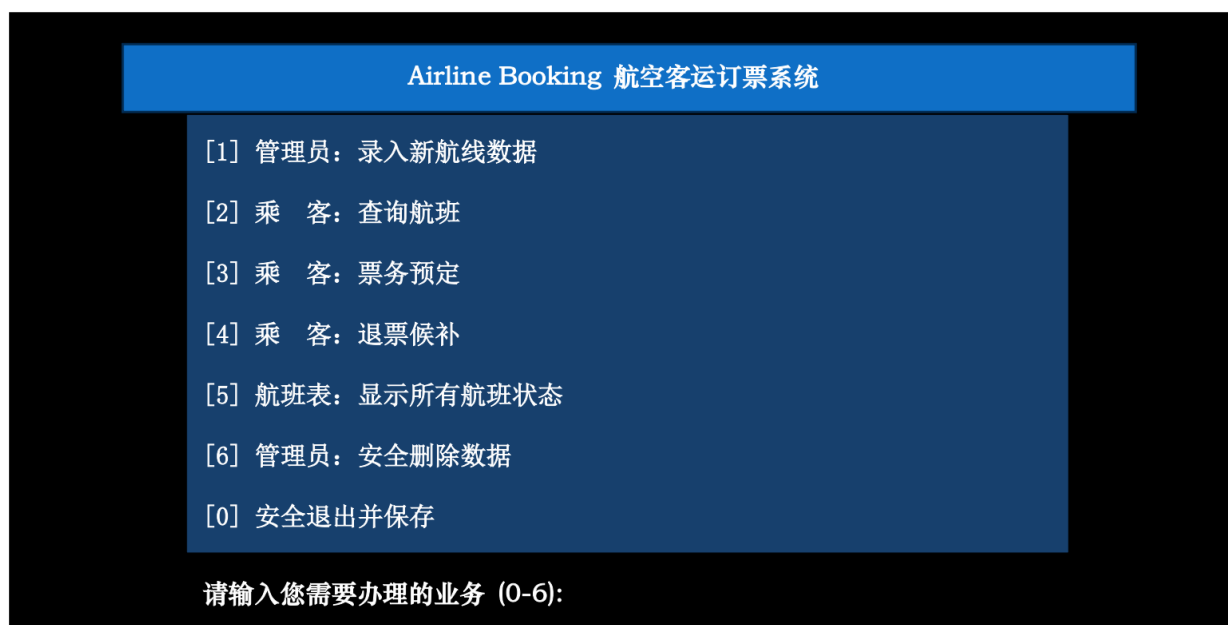


图 2 航空客运订票系统主界面的外观

【业务类型】	通过菜单栏选择，系统提示信息文字呈蓝色。
【业务内容】	业务对应详细内容输出总览，如航班票额查询、航班表等。 余票充足时票额表呈绿色，无余票时呈红色。
【用户操作栏】	用户在此输入信息。输入信息均为白色。
【系统反馈】	在此界面进行下一步操作时，界面交互提示信息呈黄色。 用户输入合法或操作成功时，文字呈绿色； 输入不合法或操作失败时，文字呈红色。

图 3 航空客运订票系统功能界面的外观

3. 重要数据的数据结构设计

航空客运订票系统的核心数据结构设计围绕航班与乘客信息的层级关系展开，采用链表作为基础数据组织形式，以实现动态数据管理和高效的节点插入及删除操作。系统主要定义了乘客信息和航班信息两类核心结构体，并通过指针构建链表实现数据的链式存储。

(1) 乘客信息链表

乘客信息链表 typedef struct Customer 通过结构体指针 struct Customer* next 串联乘客信息。

乘客信息链表具体分为已订票乘客 CustomerPtr bookedList 和候补用户名单 CustomerPtr waitList 两种。

乘客信息结构体用于存储已订票乘客和候补乘客的关键数据，包括乘客姓名、所需票数以及指向下一乘客的指针。主要定义以下变量：

- name[50]：满足中英文姓名存储需求的姓名字符数组；
- ticketCount：支持退票和候补排队管理的整型订票数量；
- next：通过指针构建单向链表结构，使系统能够动态管理同一航班的多名乘客信息。

(2) 航班信息链表

航班信息链表 typedef struct Flight 通过结构体指针 struct Flight* next 串联航班信息。

航班信息结构体整合了航线信息、运营数据、票务状态及乘客链表等多维度信息。主要定义以下变量：

- startCity、endCity：机场起降地点数组；
- flightNumber[10]：航班号数组；
- dayOfWeek[10]、time[10]：运营时间数组，包括飞行周日和航行时刻；

capacity、remainingTickets、price: 票务信息整型数据, 包括总乘员、余票量和票价;
CustomerPtr bookedList、CustomerPtr waitList: 乘客信息链表。

4. 函数清单

航空客运订票系统采用结构化程序设计思想, 由 5 个 .h 文件和 1 个 .c 文件组成。本应用程序将订票系统核心功能拆解为独立函数, 通过函数间的逻辑调用实现完整业务流程。系统函数按功能可分为初始化与界面控制函数、核心业务函数和数据管理函数三大类。具体的函数功能设计及处理描述详见表 1。

表 1 函数清单

文件名	函数原型	函数功能	处理描述
Airline Booking.c	void initConsole()	控制台初始化	设置窗口标题为“航空客运订票系统”, 调整窗口大小为 100 列×30 行, 清除屏幕内容。
	void setColor(int color)	设置文字颜色	通过 Windows API 函数 SetConsoleTextAttribute 实现文本颜色控制, 支持多功能分区交互场景。
	void printHeader(const char* title)	打印带颜色标题	结合 printSeparator() 和 setColor() 函数, 生成带分隔线的彩色标题栏, 增强界面层次感和美观性。
	void printSeparator()	打印分隔线	输出由“=”组成的分隔线, 用于功能模块间的视觉区分。
	void addFlight()	添加航班	①通过控制台输入航班详细信息, 如起点站、终点站等。 ②动态分配 Flight 结构体内存, 初始化乘客链表指针为 NULL。 ③将新航班节点插入 flightList 链表头部。
	void searchFlight()	查询航班	①根据用户输入的出发地和目的地, 遍历 flightList 链表。 ②匹配航班信息后, 按余票状态动态调整输出颜色。 ③输出结果包含航班号、飞行时间、余票量等关键信息。
	void bookTicket()	办理订票	①验证用户输入的航班号、姓名及订票数量合法性。 ②若余票充足, 直接创建乘客节点并插入 bookedList 链表。 ③若余票不足, 推荐同航线其他航班或根据用户选择插入 waitList 候补队列。

	<code>void returnTicket()</code>	办理退票	①验证乘客退票资格，遍历 <code>bookedList</code> 链表查找目标乘客。 ② 释 放 退 票 对 应 的 余 票 ， 更 新 <code>remainingTickets</code> 。 ③按先进先出原则检查 <code>waitList</code> 队列，自动为候补乘客办理补票
	<code>void saveToFile()</code>	保存数据到文件	<code>saveToFile()</code> 通过二进制写入将 <code>flightList</code> 链表数据保存至 <code>flights.dat</code> 文件。
	<code>void loadFromFile()</code>	从文件加载数据	<code>loadFromFile()</code> 在程序启动时从文件恢复数据，重建航班与乘客链表。
	<code>void deleteFlight()</code>	删除航班	①显示所有航班列表供管理员选择。 ②二次确认后从 <code>flightList</code> 链表中移除目标节点。 ③ 递 归 释 放 该 航 班 的 <code>bookedList</code> 和 <code>waitList</code> 链表内存，防止内存泄漏。
	<code>void displayAllFlights()</code>	显示所有航班	遍历 <code>flightList</code> 链表，格式化输出所有航班的核心信息（航班号、航线、余票量等），支持管理员全局监控。
	<code>void deleteSavedFlight()</code>	安全删除数据	通过序号选择实现航班的精确删除，结合多重确认流程防止误操作，确保数据安全性。

五、源程序

整个航空客运订票系统应用程序由 1 个源文件 `Airline Booking.c` 构成。
源程序见附录一。

六、测试

1. 主界面菜单指令测试

检查系统主界面布局是否合理、正确、清晰。首先，用户在操作栏输入不合法的业务序号，观察系统是否提示错误，校验应用程序有效性。其次，用户输入合法的业务序号，观察系统是否跳转到相应功能界面。测试过程如图 4 至图 6 所示。

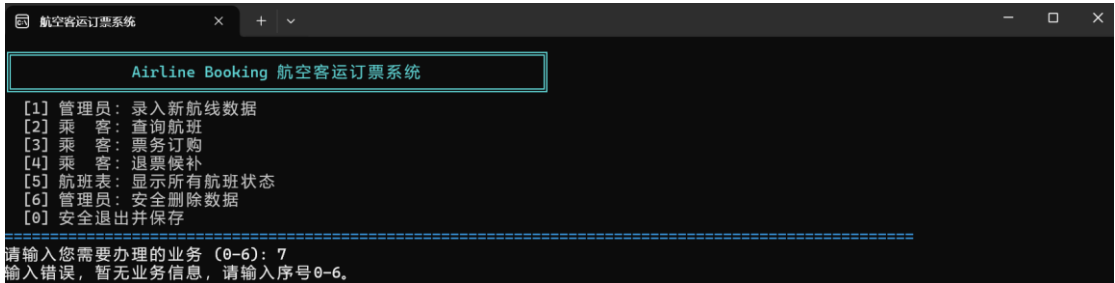


图 4 主界面输入序号不合法

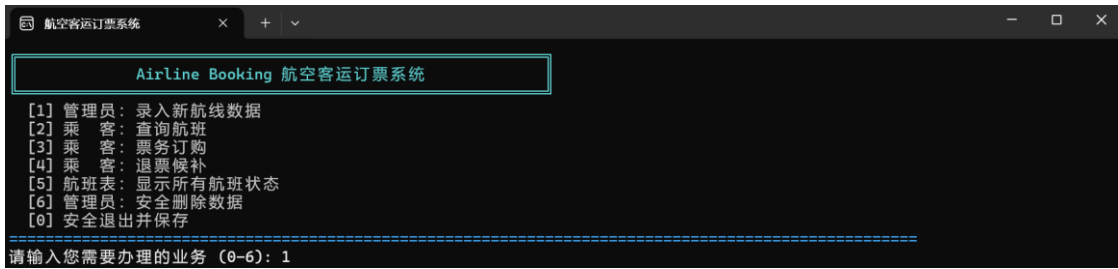


图 5 主界面输入序号合法

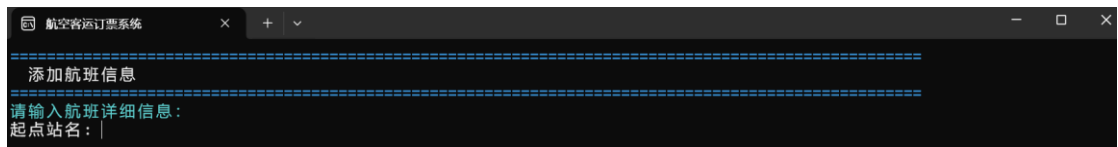


图 6 主界面跳转至功能界面

结论：界面布局合理、正确、清晰，应用程序有效。输入不合法序号后，系统报错且干扰后续功能；输入合法序号后，系统正常跳转到相应功能界面。

2. 航空客运订票系统的各子功能测试

(1) 录入新航线数据的测试

用户在主界面操作栏输入序号 1 后，正常跳转到该功能界面，通过回车键换行输入该航班全部信息。测试过程如图 7 所示。

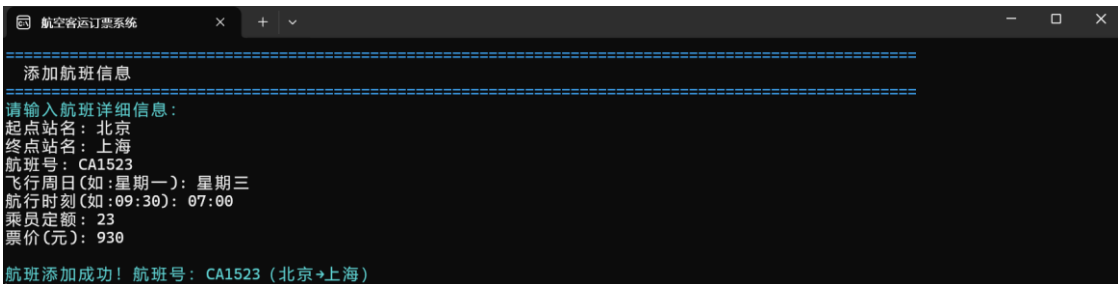


图 7 成功录入新航线数据

结论：录入新航线数据功能已按预期实现。

(2) 显示所有航班状态的测试

用户在主界面操作栏输入序号 5 后，正常跳转到该功能界面，航班表显示订票系统已录入的全部航班信息。测试过程如图 8 所示。

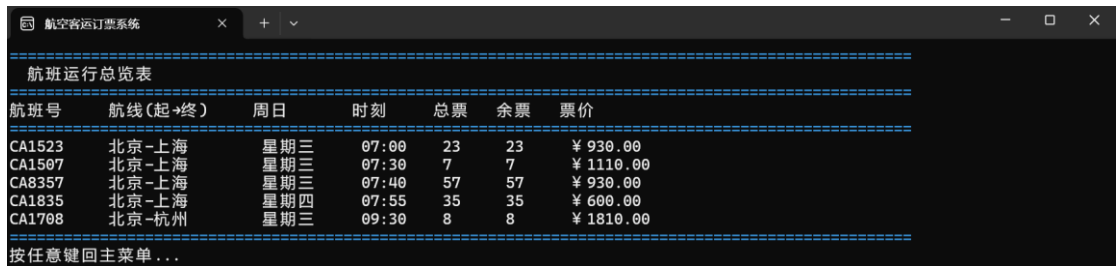


图 8 成功显示所有航班状态

结论：显示所有航班状态功能已按预期实现。

(3) 查询航班的测试

用户在主界面操作栏输入序号 2 后，正常跳转到该功能界面。

输入航线合法时，航班表显示符合用户需求的航班信息；输入航线不合法时，航班表反馈未查询到目标航线。同时，测试应观察航班表实时变化情况，有余票时航班呈绿色，无余票时航班呈红色。

本次测试的航班表实时变化情况，选取了初始查询结果和执行过第（4）、（5）项测试后的查询结果进行对比。测试过程如图 9 至图 11 所示。

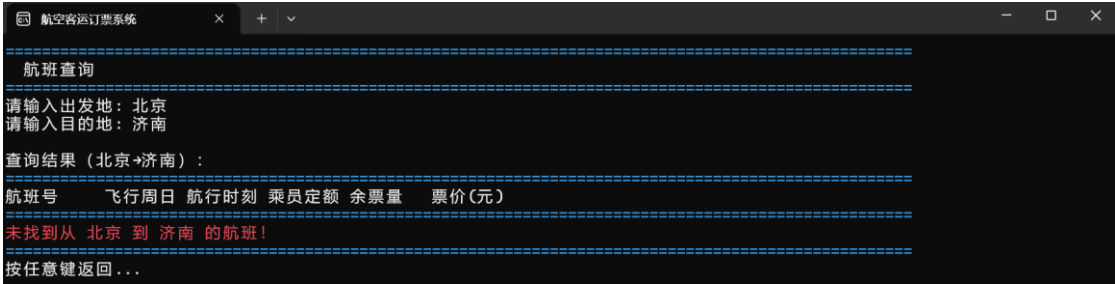


图 9 输入航线不合法

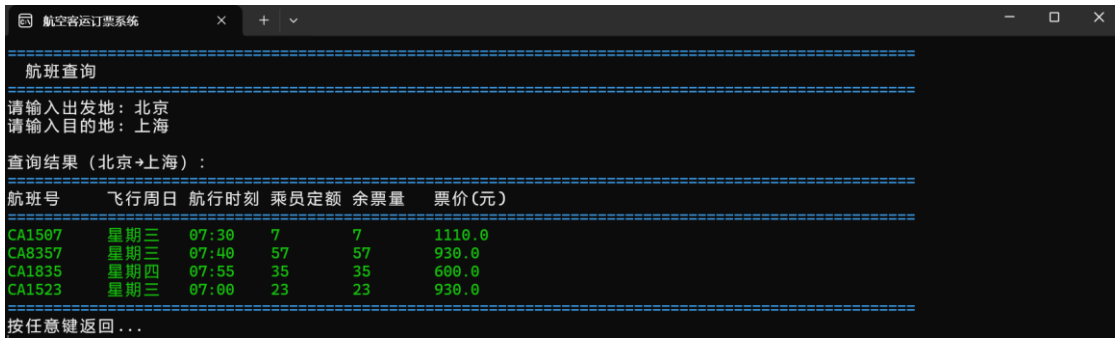


图 10 输入航线合法

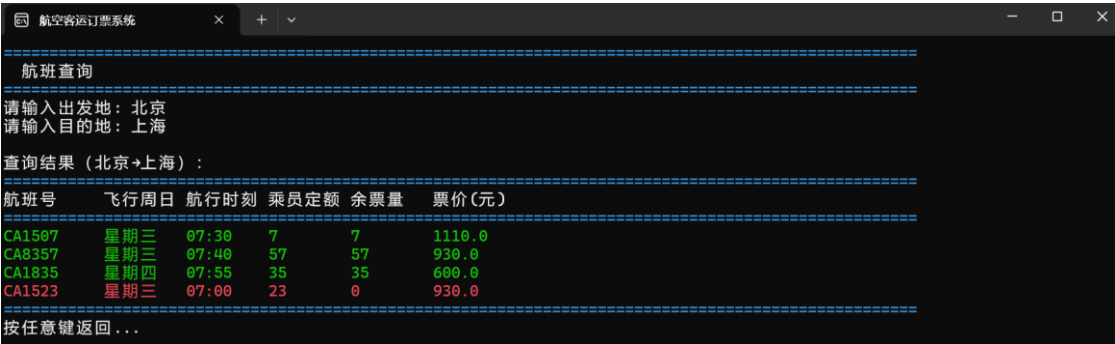


图 11 成功显示航班表实时变化情况

结论：应用程序能根据用户需求反馈实时变化的航班信息。查询航班功能已按预期实现。

(4) 票务订购的测试

用户在主界面操作栏输入序号 3 后，正常跳转到该功能界面。用户输入信息后，观察应

用程序是否能判断信息合法性。输入信息不合法时，系统应给予红字警示；输入信息合法时，系统按照用户指令进行下一步业务办理流程。

本次测试以航班 CA1523 为例，该班次总计 23 张机票。已知小 A 需订购 2 张，小 B 需订购 23 张，小 C 需订购 22 张。订票先后顺序为小 A、小 B、小 C。测试过程如图 12 至图 18 所示。

测试情景如下：

订票航班输入不合法时，订票失败。

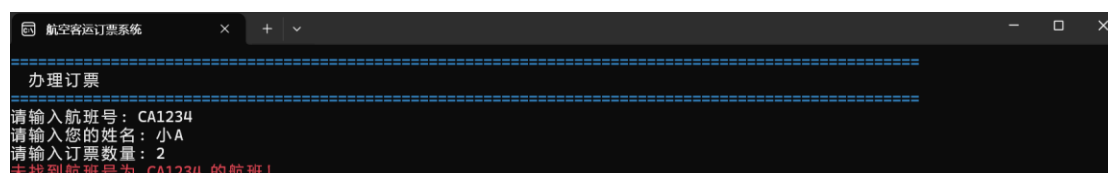


图 12 订票航班输入不合法

订票数量输入不合法时，订票失败。

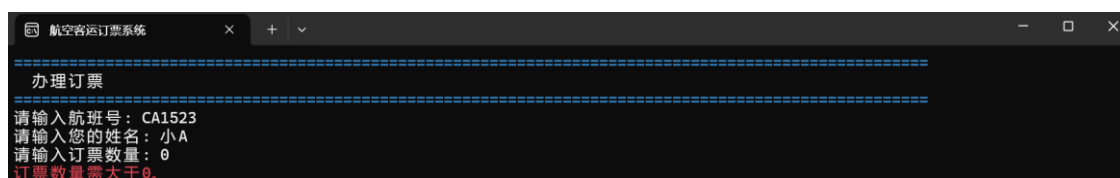


图 13 订票数量输入不合法

小 A 订票时，余票充足，订票成功。

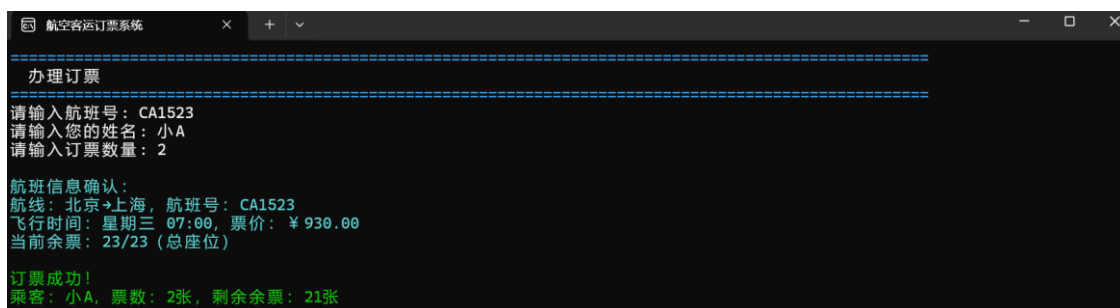


图 14 余票充足成功订票

小 B 订票时，余票不充足，若他选择更换航班，则可根据系统推荐重新自主订票。

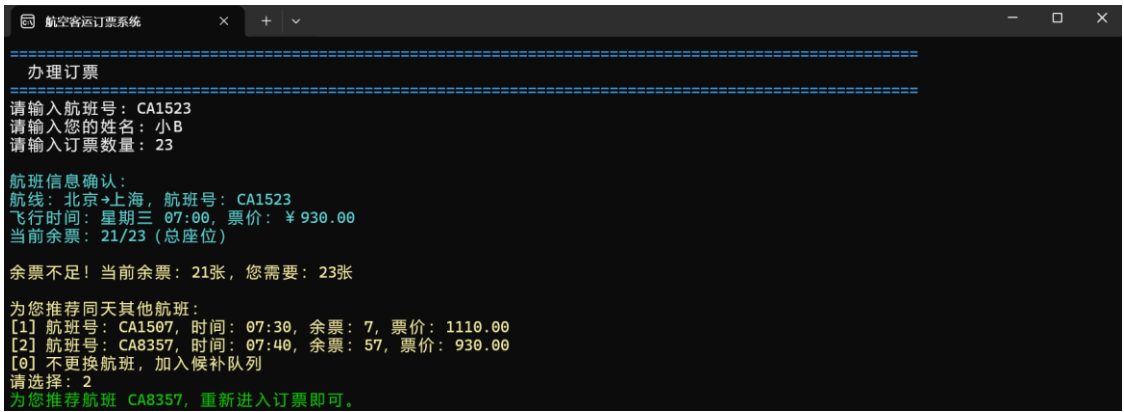


图 15 余票不足系统推荐新航班

若小 B 选择不更换航班，则系统推荐候补。此时，小 B 选择加入候补列队。

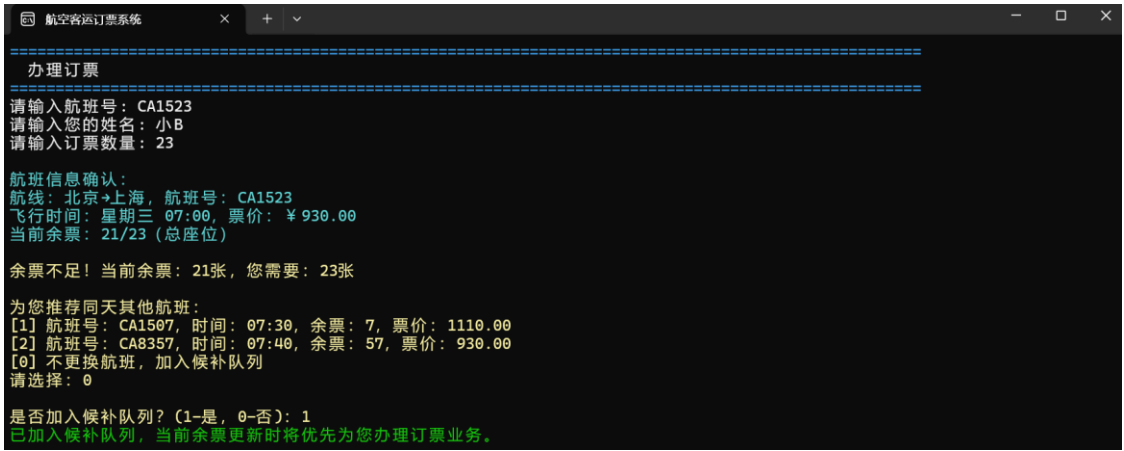


图 16 余票不足加入候补（例 1）

小 C 订票时，余票不充足，若他选择不更换航班且不候补，则订票失败。

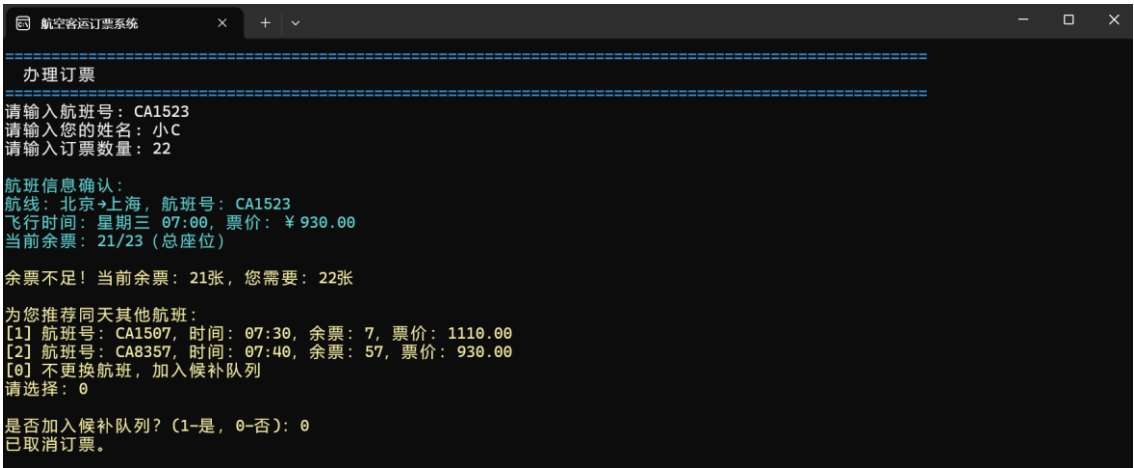


图 17 余票不足不候补

最终，小 C 选择加入候补。

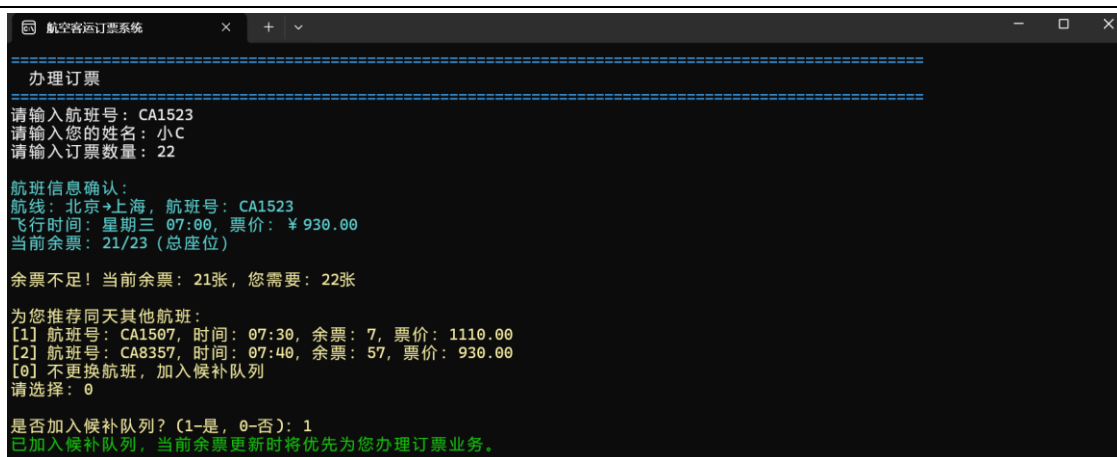


图 18 余票不足加入候补（例 2）

结论：应用程序能判断输入信息的合法性，正常警示，执行办理订票、加入候补等业务流程。票务订购功能已按预期实现。

（5）退票候补的测试

用户在主界面操作栏输入序号 4 后，正常跳转到该功能界面。用户输入信息后，观察应用程序是否能判断信息合法性，并根据不同退票数量做出相应反馈。输入信息不合法时，系统应给予红字警示；输入信息合法时，系统按照用户指令进行下一步业务办理流程。

应用程序按照先进先出原则处理候补用户，若退票数量满足队列首位用户的订票数额需求，则自动完成票务转移；否则，将依次询问后续候补用户。测试需观察程序是否能严格按照该逻辑执行候补。

本次测试主体以航班 CA1523 为例，该班次总计 23 张机票。已知已订票链表里小 A 订 2 张，小 A 需要部分退票（仅退 1 张），候补链表里小 B 需订 23 张、小 C 需订 22 张。候补排序为小 B、小 C。

同时，本次测试单独补充了小 A 需全部退票（退票 2 张）的情况。测试过程如图 19 至图 24 所示。

测试情景如下：

退票航班输入不合法时，退票失败。

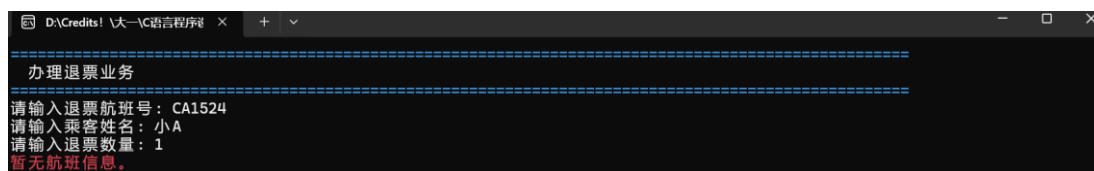


图 19 退票航班输入不合法

票额输入信息为非正数或超额时，均退票失败。

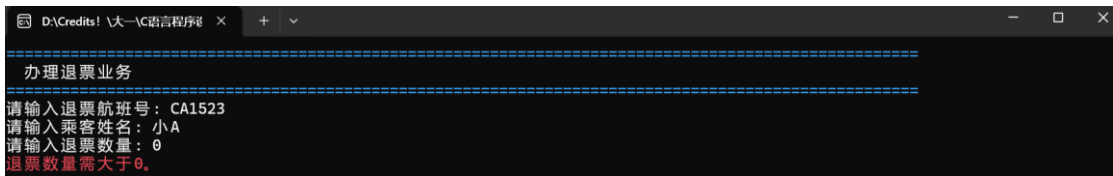


图 20 退票票额输入非正数

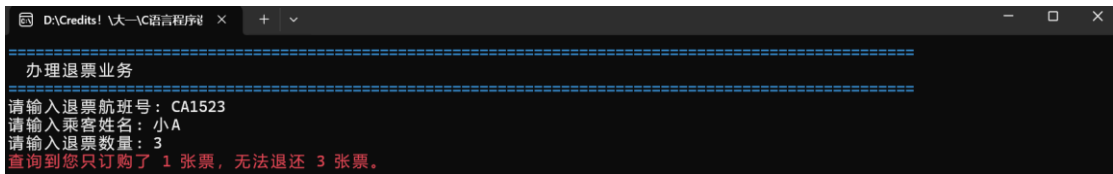


图 21 退票票额输入超额

订票姓名输入不合法时，退票失败。

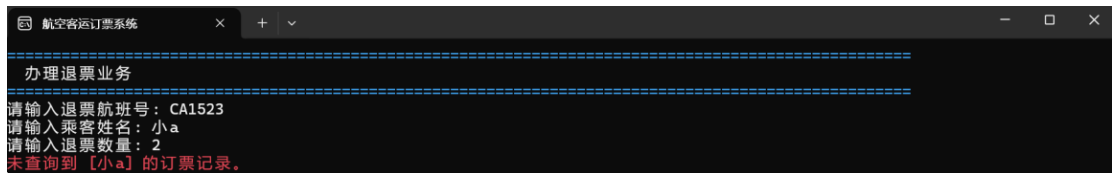


图 22 订票姓名输入不合法

小 A 正确输入全部信息后，退票成功。同时，系统自动检索候补乘客链表。新余票量不满足小 B 订票需求，小 B 候补失败；新余票量满足小 C 订票需求，小 C 候补成功。

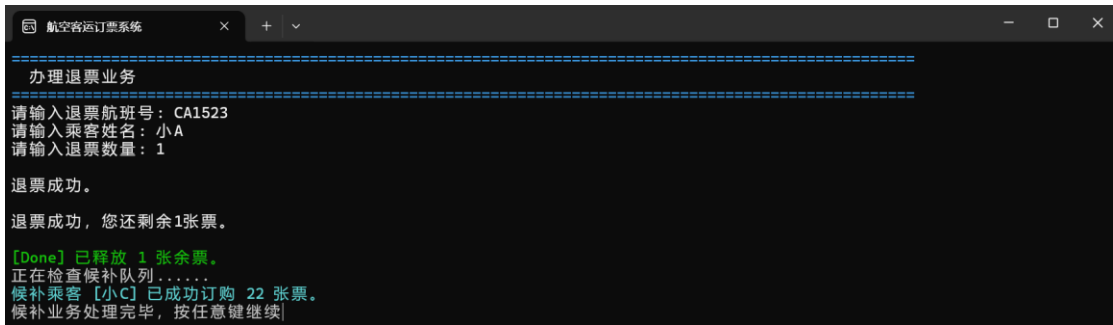


图 23 部分退票操作成功且自动候补

补充测试：小 A 退票 2 张，即全部退票时，系统应更换反馈文字。

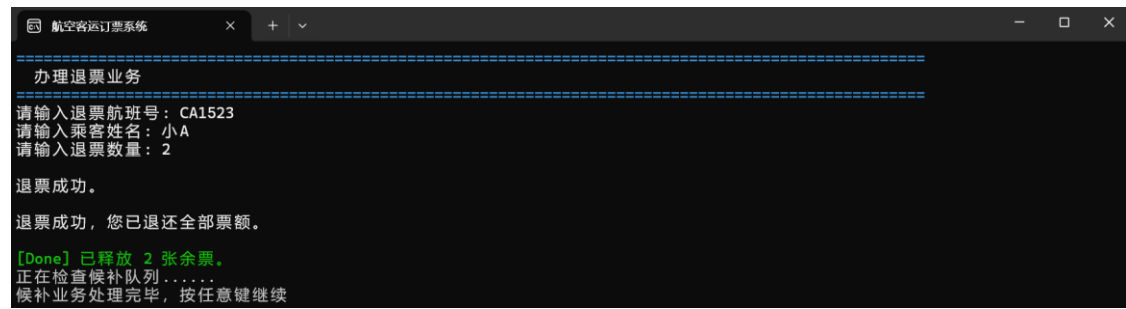


图 24 全部退票操作成功

结论：应用程序能判断输入信息的合法性，正常警示，执行办理退票、自动候补等业务流程，并根据不同退票数量做出相应反馈。退票候补功能已按预期实现。

(6) 安全删除数据的测试

用户在主界面操作栏输入序号 5 后，正常跳转到该功能界面。用户输入信息后观察应用程序是否能判断信息合法性，并根据不同选项序号做出相应反馈。输入序号不合法时，系统应取消删除操作，保护数据；输入信息合法时，系统需向用户二次确认删除操作，防止误删数据。测试过程如图 25 至图 26 所示。

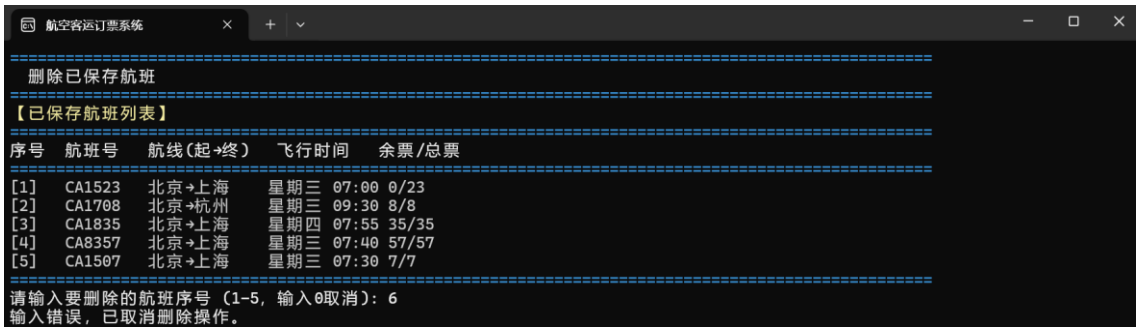


图 25 删除序号输入不合法

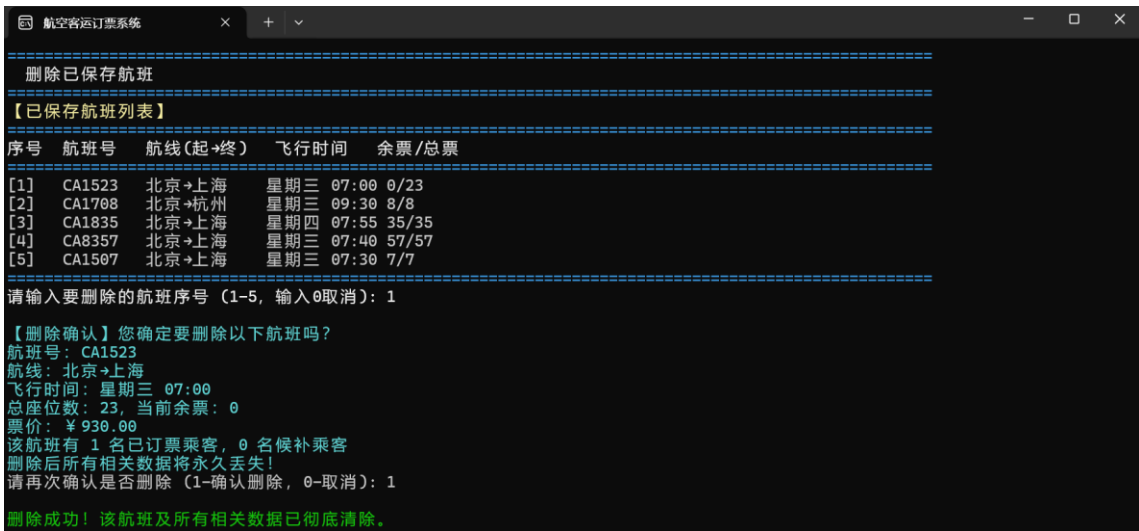


图 26 成功安全删除数据

结论：应用程序能判断操作合法性，一定程度上保护数据并防止误删操作。安全删除数据功能已按预期实现。

综上，通过测试可以基本确定航空客运订票系统实现了预定的功能。

七、参考文献

- [1] 何钦铭, 颜晖 《C 语言程序设计》，高等教育出版社.
- [2] 何钦铭, 陈根才 《C 语言程序设计课程设计》，浙江大学出版社.
- [3] 郑玲 《C 语言程序设计》，中国电力出版社.
- [4] 谭浩强 《C 语言设计（第五版）》，清华大学出版社.
- [5] Kenneth A. Reek. 《C 和指针》，人民邮电出版社.
- [6] 林锐 《高质量 C/C++编程指南》，电子工业出版社.
- [7] 吴岳良 《C 语言编程实战》，机械工业出版社.
- [8] 张莉, 王春玲 《C 语言程序设计案例教程》，人民邮电出版社.
- [9] 夏宽理, 赵志宏 《C 语言程序设计教程》，复旦大学出版社.

附录一：源程序

```
#define _CRT_SECURE_NO_WARNINGS // Visual Studio 消除 scanf/strcpy 安全警告
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <conio.h> // 用于_getch()函数
#include <windows.h> // 用于控制台颜色控制

// 结构体
// 乘客信息结构体
typedef struct Customer {
    char name[50]; // 乘客姓名
    int ticketCount; // 所需票数
    struct Customer* next; // 指向下一个乘客（乘客链表）
} Customer, * CustomerPtr;

// 航班信息结构体
typedef struct Flight {
    // 1. 起降机场信息
    char startCity[50]; // 起点站名（如“北京”）
    char endCity[50]; // 终点站名（如“上海”）

    // 2. 航班标识信息
    char flightNumber[10]; // 航班号（如“CA1234”）

    // 3. 运营时间信息
    char dayOfWeek[10]; // 飞行周日（如“星期一”）
    char time[10]; // 航行时刻（如“09:30”）

    // 4. 票务信息
    int capacity; // 乘员定额（总座位数）
    int remainingTickets; // 当前余票量
    float price; // 票价（单位：元）

    // 5. 乘客信息（乘客链表）
    CustomerPtr bookedList; // 已订票乘客名单
    CustomerPtr waitList; // 候补用户名单

    struct Flight* next; // 指向下一个航班（航班链表）
} Flight, * FlightPtr;

// 全局变量
Flight* flightList = NULL; // 航班链表头指针
HANDLE hConsole; // 控制台句柄（控制颜色）
```



```
// 函数声明
void initConsole();           // 初始化控制台
void printMenu();             // 打印主菜单
void printSeparator();        // 打印分隔线
void setColor(int color);     // 设置文字颜色
void printHeader(const char* title); // 打印带颜色的标题

// 核心功能函数
void addFlight();             // 添加航班
void deleteFlight();          // 删除航班
void searchFlight();          // 查询航班
void bookTicket();            // 办理订票
void returnTicket();          // 办理退票
void displayAllFlights();     // 显示所有航班
void saveToFile();            // 保存数据到文件
void loadFromFile();          // 从文件加载数据
void deleteSavedFlight();     // 安全删除数据

// 工具函数
// 设置字体颜色
void setColor(int color) {
    hConsole = GetStdHandle(STD_OUTPUT_HANDLE);
    SetConsoleTextAttribute(hConsole, color);
}

// 打印分隔线
void printSeparator() {
    setColor(3);
    for (int i = 0; i < 100; i++) printf("=");
    printf("\n");
    setColor(15);
}

// 打印带颜色的标题
void printHeader(const char* title) {
    system("cls");
    printSeparator();
    setColor(15);
    printf(" %s \n", title);
    printSeparator();
    setColor(15);
}

// 初始化控制台
void initConsole() {
    hConsole = GetStdHandle(STD_OUTPUT_HANDLE);
```

```
system("title 航空客运订票系统"); // 设置窗口标题
system("mode con cols=100 lines=30"); // 设置窗口大小
system("cls"); // 清屏
}

// 一、录入新航线信息
void addFlight() {
    printHeader("添加航班信息");

    // 创建新航班节点
    Flight* newFlight = (Flight*)malloc(sizeof(Flight));
    if (newFlight == NULL) {
        setColor(12); // 红色错误提示
        printf("内存分配失败！无法添加航班。\\n");
        setColor(15);
        _getch(); // 按任意键返回（_getch 避免报错）
        return;
    }

    // 初始化新航班
    newFlight->bookedList = NULL;
    newFlight->waitList = NULL;
    newFlight->next = NULL;

    setColor(11);
    printf("请输入航班详细信息：\\n");
    setColor(15);

    printf("起点站名：");
    scanf("%s", newFlight->startCity);
    printf("终点站名：");
    scanf("%s", newFlight->endCity);
    printf("航班号：");
    scanf("%s", newFlight->flightNumber);
    printf("飞行周日(如:星期一)：");
    scanf("%s", newFlight->dayOfWeek);
    printf("航行时刻(如:09:30)：");
    scanf("%s", newFlight->time);
    printf("乘员定额：");
    scanf("%d", &newFlight->capacity);
    newFlight->remainingTickets = newFlight->capacity; // 初始余票=总座位
    printf("票价(元)：");
    scanf("%f", &newFlight->price);

    // 添加到航班链表头部
    newFlight->next = flightList;
```

```
flightList = newFlight;

setColor(11); // 成功提示
printf("\n 航班添加成功! 航班号: %s ( %s → %s ) \n", newFlight->flightNumber,
newFlight->startCity, newFlight->endCity);
setColor(15);
_getch(); // 按任意键返回
}

//二、乘客查询航班
void searchFlight() {
    printHeader("航班查询");

    // 未录入航班信息
    if (flightList == NULL) {
        setColor(12);
        printf("暂无航班信息! \n");
        setColor(15);
        _getch();
        return;
    }

    // 查询
    char startCity[50], endCity[50];
    printf("请输入出发地: ");
    scanf("%s", startCity);
    printf("请输入目的地: ");
    scanf("%s", endCity);

    // 打印查询结果表头
    printf("\n");
    setColor(15);
    printf("查询结果 (%s→%s): \n", startCity, endCity);
    printSeparator();
    printf("%-10s %-8s %-8s %-8s %-8s %-8s\n", "航班号", "飞行周日", "航行时刻", "乘员定
额", "余票量", "票价(元)");
    printSeparator();
    setColor(15);

    Flight* current = flightList;
    int found = 0;

    // 遍历航班链表, 查找匹配的航班
    while (current != NULL) {
        if (strcmp(current->startCity, startCity) == 0 && strcmp(current->endCity,
endCity) == 0) {
```

```
        found = 1;
        // 根据余票量显示不同颜色
        if (current->remainingTickets == 0) setColor(12); // 无票红色
        else setColor(10); // 余票充足绿色

        // 按需求输出所有查询字段
        printf("%-10s %-8s %-8s %-8d %-8d %-8.1f\n",
                current->flightNumber,
                current->dayOfWeek,
                current->time,
                current->capacity,
                current->remainingTickets,
                current->price);
    }
    current = current->next;
}

if (!found) {
    setColor(12);
    printf("未找到从 %s 到 %s 的航班! \n", startCity, endCity);
}
setColor(7);
printSeparator();
printf("按任意键返回... \n");
_getch();
}

// 三、乘客办理订票：订票功能 + 余票判断 + 候补推荐
void bookTicket() {
    printHeader("办理订票");

    char flightNum[10], name[50];
    int ticketCount;
    printf("请输入航班号: ");
    scanf("%s", flightNum);
    printf("请输入您的姓名: ");
    scanf("%s", name);
    printf("请输入订票数量: ");
    scanf("%d", &ticketCount);

    // 验证订票合法性
    if (flightList == NULL) {
        setColor(12);
        printf("暂无航班信息。 \n");
        setColor(7);
        _getch();
    }
}
```

```
        return;
    }

    if (ticketCount <= 0) {
        setColor(12);
        printf("订票数量需大于 0。 \n");
        setColor(7);
        _getch();
        return;
    }

    Flight* targetFlight = NULL;
    Flight* current = flightList;
    while (current != NULL) {
        if (strcmp(current->flightNumber, flightNum) == 0) {
            targetFlight = current;
            break;
        }
        current = current->next;
    }

    if (targetFlight == NULL) {
        setColor(12);
        printf("未找到航班号为 %s 的航班! \n", flightNum);
        setColor(7);
        _getch();
        return;
    }

    // 确认航班信息
    setColor(11);
    printf("\n 航班信息确认: \n");
    printf("航线: %s→%s, 航班号: %s\n", targetFlight->startCity, targetFlight->endCity,
targetFlight->flightNumber);
    printf("飞行时间: %s %s, 票价: ￥%.2f\n", targetFlight->dayOfWeek, targetFlight->time,
targetFlight->price);
    printf("当前余票: %d/%d (总座位) \n", targetFlight->remainingTickets,
targetFlight->capacity);
    setColor(7);

    // 判断余票是否充足
    if (ticketCount <= targetFlight->remainingTickets) {
        // 余票充足, 直接订票
        Customer* newCustomer = (Customer*)malloc(sizeof(Customer));
        if (newCustomer == NULL) {
            setColor(12);
```

```
        printf("内存分配失败, 订票失败! \n");
        setColor(7);
        _getch();
        return;
    }
    strcpy(newCustomer->name, name);
    newCustomer->ticketCount = ticketCount;
    newCustomer->next = targetFlight->bookedList;
    targetFlight->bookedList = newCustomer;

    targetFlight->remainingTickets -= ticketCount;

    setColor(10);
    printf("\n 订票成功! \n");
    printf(" 乘客: %s, 票数: %d 张, 剩余余票: %d 张 \n", name, ticketCount,
targetFlight->remainingTickets);
    setColor(7);
}
else {
    // 余票不足, 先推荐同天、同路线其他航班
    setColor(14);
    printf("\n 余票不足!当前余票:%d张,您需要:%d张\n", targetFlight->remainingTickets,
ticketCount);

    // 查找航班
    Flight* alternatives[10];
    int altCount = 0;
    current = flightList;
    while (current != NULL && altCount < 10) {
        if (strcmp(current->dayOfWeek, targetFlight->dayOfWeek) == 0 &&
            strcmp(current->startCity, targetFlight->startCity) == 0 &&
            strcmp(current->endCity, targetFlight->endCity) == 0 &&
            strcmp(current->flightNumber, targetFlight->flightNumber) != 0 &&
            current->remainingTickets > 0) {
            alternatives[altCount++] = current;
        }
        current = current->next;
    }

    // 推荐航班
    // 有可推荐航班
    if (altCount > 0) {
        printf("\n 为您推荐同天其他航班: \n");
        for (int i = 0; i < altCount; i++) {
            printf("[%d] 航班号: %s, 时间: %s, 余票: %d, 票价: %.2f\n", i + 1,
alternatives[i]->flightNumber, alternatives[i]->time, alternatives[i]->remainingTickets,
```

```
alternatives[i]->price);
    }
    printf("[0] 不更换航班, 加入候补队列\n");
    printf("请选择: ");

    int choice;
    scanf("%d", &choice);
    if (choice > 0 && choice <= altCount) {
        // 选择推荐航班, 提示重新订票
        setColor(10);
        printf("为您推荐航班 %s, 重新进入订票即可.\n", alternatives[choice -
1]->flightNumber);
        setColor(7);
        _getch();
        return;
    }
}
// 无可推荐航班
else {
    printf("\n 当天无其他同路线航班可推荐.\n");
}

// 是否候补
printf("\n 是否加入候补队列? (1-是, 0-否): ");
int waitChoice;
scanf("%d", &waitChoice);

// 参与候补: 加入链表
if (waitChoice == 1) {
    Customer* newWaiter = (Customer*)malloc(sizeof(Customer));
    if (newWaiter == NULL) {
        setColor(12);
        printf("内存分配失败! \n");
        setColor(7);
        _getch();
        return;
    }
    strcpy(newWaiter->name, name);
    newWaiter->ticketCount = ticketCount;
    newWaiter->next = NULL;

    // 加入候补链表
    if (targetFlight->waitList == NULL) {
        targetFlight->waitList = newWaiter;
    }
    else {
```

```
        Customer* last = targetFlight->waitList;
        while (last->next != NULL) last = last->next;
        last->next = newWaiter;
    }

    setColor(10);
    printf("已加入候补队列，当前余票更新时将优先为您办理订票业务。\\n");
    setColor(7);
}
// 拒绝候补：取消订票
else {
    printf("已取消订票。\\n");
}
}
_getch();
}
```

// 四、乘客办理退票：资格验证 + 候补自动补位

```
void returnTicket() {
    printHeader("办理退票业务");

    if (flightList == NULL) {
        setColor(12);
        printf("暂无航班信息。\\n");
        setColor(7);
        _getch();
        return;
    }

    char flightNum[10], name[50];
    int returnCount;
    printf("请输入退票航班号：");
    scanf("%s", flightNum);
    printf("请输入乘客姓名：");
    scanf("%s", name);
    printf("请输入退票数量：");
    scanf("%d", &returnCount);

    if (returnCount <= 0) {
        setColor(12);
        printf("退票数量需大于 0。\\n");
        setColor(7);
        _getch();
        return;
    }
}
```



```
// 寻找目标航班
FlightPtr targetFlight = flightList;
while (targetFlight != NULL && strcmp(targetFlight->flightNumber, flightNum) != 0) {
    targetFlight = targetFlight->next;
}

if (targetFlight == NULL) {
    setColor(12);
    printf("暂无航班信息。\\n", flightNum);
    setColor(7);
    _getch();
    return;
}

// 验证退票资格：遍历已订票乘客链表
CustomerPtr curr = targetFlight->bookedList;
CustomerPtr prev = NULL;
int success = 0;

while (curr != NULL) {
    if (strcmp(curr->name, name) == 0) {
        if (curr->ticketCount >= returnCount) {
            targetFlight->remainingTickets += returnCount;
            curr->ticketCount -= returnCount;
            success = 1;
            printf("\\n 退票成功。\\n");

            // 若该乘客退票至 0，将其从链表剔除
            if (curr->ticketCount == 0) {
                CustomerPtr toDelete = curr;    //记录要删除的乘客节点
                if (prev == NULL) targetFlight->bookedList = curr->next;
                else prev->next = curr->next;
                free(toDelete);    // 先指向下一个乘客，再释放内存！！！！
                printf("\\n 退票成功，您已退还全部票额。\\n");
            }
            else {
                printf("\\n 退票成功，您还剩余%d 张票。\\n", curr->ticketCount);
            }
            break;
        }
        else {
            setColor(12);
            printf(" 查询到您只订购了   %d   张票，无法退还   %d   张票。\\n",
curr->ticketCount, returnCount);
            setColor(7);
            _getch();
        }
    }
    prev = curr;
    curr = curr->next;
}
```

```
        return;
    }
}

prev = curr;
curr = curr->next;
}

// 找不到该乘客
if (!success) {
    setColor(12);
    printf("未查询到 [%s] 的订票记录。\\n", name);
    setColor(7);
    _getch();
    return;
}

// 执行退票
setColor(10);
printf("\\n[Done] 已释放 %d 张余票。\\n", returnCount);
setColor(7);

// 候补自动补位逻辑：有余票 + 有候补
printf("正在检查候补队列.....\\n");
CustomerPtr wCurr = targetFlight->waitList;
CustomerPtr wPrev = NULL;

while (wCurr != NULL && targetFlight->remainingTickets > 0) {

    //若该候补乘客需票量不大于余票
    if (wCurr->ticketCount <= targetFlight->remainingTickets) {
        setColor(11);
        printf("候补乘客 [%s] 已成功订购 %d 张票。\\n", wCurr->name,
wCurr->ticketCount);

        // 更新余票
        targetFlight->remainingTickets -= wCurr->ticketCount;

        // 从候补链表移除该乘客节点
        CustomerPtr toTransfer = wCurr;
        if (wPrev == NULL) targetFlight->waitList = wCurr->next;
        else wPrev->next = wCurr->next;

        // 将该乘客加入已订票链表
        toTransfer->next = targetFlight->bookedList;
        targetFlight->bookedList = toTransfer;
    }
}
```

```
        // 移动指针到下一个候补：指针重置
        if (wPrev == NULL) wCurr = targetFlight->waitList;
        else wCurr = wPrev->next;
        setColor(7);
    }
    else {
        // 若该候补乘客需票量大于余票，按顺序检索下一位候补乘客
        wPrev = wCurr;
        wCurr = wCurr->next;
    }
}
printf("候补业务处理完毕，按任意键继续");
_getch();
}
```

// 五、航班表

```
void displayAllFlights() {
    printHeader("航班运行总览表");
    if (flightList == NULL) {
        printf("暂无航班信息。\\n");
        _getch();
        return;
    }

    printf("%-10s %-16s %-10s %-8s %-6s %-6s %-10s\\n",
        "航班号", "航线(起→终)", "周日", "时刻", "总票", "余票", "票价");
    printSeparator();

    FlightPtr f = flightList;
    while (f != NULL) {
        char route[40];
        sprintf(route, "%s-%s", f->startCity, f->endCity);
        printf("%-10s %-16s %-10s %-8s %-6d %-6d ￥%.2f\\n", f->flightNumber, route,
            f->dayOfWeek, f->time, f->capacity, f->remainingTickets, f->price);
        f = f->next;
    }
    printSeparator();
    printf("按任意键回主菜单...");
    _getch();
}
```

// 六、安全删除航班数据：安全验证 + 内存释放

```
void deleteSavedFlight() {
    printHeader("删除已保存航班");

    // 检查是否有航班可删除
```

```
if (flightList == NULL) {
    setColor(12);
    printf("暂无航班信息。\\n");
    setColor(7);
    _getch();
    return;
}

// 显示所有已保存航班，管理员选择需删除的航班
setColor(14);
printf("【已保存航班列表】\\n");
printSeparator();
printf("序号   航班号   航线(起→终)   飞行时间   余票/总票\\n");
printSeparator();
setColor(7);

// 临时数组存储航班指针：按序号删除选项
FlightPtr flights[100];
int flightCount = 0;
FlightPtr current = flightList;

//存储航班指针
while (current != NULL && flightCount < 100) {
    flights[flightCount] = current;
    char route[40];
    sprintf(route, "%s→%s", current->startCity, current->endCity);
    printf("[%d]   %-8s %-13s %s %-4s %d/%d\\n",
        flightCount + 1,
        current->flightNumber,
        route,
        current->dayOfWeek,
        current->time,
        current->remainingTickets,
        current->capacity);
    current = current->next;
    flightCount++;
}

printSeparator();
printf("请输入要删除的航班序号 (1-%d, 输入 0 取消): ", flightCount);
int choice;
scanf("%d", &choice);

// 验证选择合法性
if (choice < 0 || choice > flightCount) {
    printf("输入错误，已取消删除操作。\\n");
```

```
        _getch();
        return;
    }
    if (choice == 0) {
        printf("已取消删除操作。\\n");
        _getch();
        return;
    }

    // 获取要删除的航班指针
    FlightPtr targetFlight = flights[choice - 1];

    // 二次确认，防止误删
    setColor(11);
    printf("\\n【删除确认】您确定要删除以下航班吗? \\n");
    printf("航班号: %s\\n", targetFlight->flightNumber);
    printf("航线: %s->%s\\n", targetFlight->startCity, targetFlight->endCity);
    printf("飞行时间: %s %s\\n", targetFlight->dayOfWeek, targetFlight->time);
    printf("总座位数: %d, 当前余票: %d\\n", targetFlight->capacity,
targetFlight->remainingTickets);
    printf("票价: ￥%.2f\\n", targetFlight->price);

    // 统计该航班人数
    int bookedCount = 0, waitCount = 0;
    CustomerPtr cust = targetFlight->bookedList;
    while (cust != NULL) { bookedCount++; cust = cust->next; }
    cust = targetFlight->waitList;
    while (cust != NULL) { waitCount++; cust = cust->next; }
    printf("该航班有 %d 名已订票乘客, %d 名候补乘客\\n", bookedCount, waitCount);
    printf("删除后所有相关数据将永久丢失! \\n");
    setColor(7);

    // 最终确认
    printf("请再次确认是否删除 (1-确认删除, 0-取消): ");
    int confirm;
    scanf("%d", &confirm);
    if (confirm != 1) {
        printf("已取消删除。\\n");
        _getch();
        return;
    }

    // 执行删除操作
    // 从航班链表中移除该航班节点
    FlightPtr prev = NULL, curr = flightList;
    while (curr != NULL && curr != targetFlight) {
```

```
        prev = curr;
        curr = curr->next;
    }
    // 删除头结点
    if (prev == NULL) {
        flightList = targetFlight->next;
    }
    // 删除其他节点
    else {
        prev->next = targetFlight->next;
    }

    // 释放该航班相关乘客内存，防止内存泄漏
    // 释放已订票乘客
    CustomerPtr currCust = targetFlight->bookedList;
    while (currCust != NULL) {
        CustomerPtr temp = currCust;
        currCust = currCust->next;
        free(temp);
    }
    // 释放候补乘客
    currCust = targetFlight->waitList;
    while (currCust != NULL) {
        CustomerPtr temp = currCust;
        currCust = currCust->next;
        free(temp);
    }

    // 释放航班结点的内存
    free(targetFlight);

    // 删除成功
    setColor(10);
    printf("\n 删除成功！ 该航班及所有相关数据已彻底清除。 \n");
    setColor(7);
    _getch();
}

// 零、保存数据！！！！
void saveToFile() {
    FILE* fp = fopen("flights.dat", "wb");
    if (fp == NULL) return;

    FlightPtr curr = flightList;
    while (curr != NULL) {
        fwrite(curr, sizeof(Flight), 1, fp);
    }
}
```

```
        curr = curr->next;
    }
    fclose(fp);
    setColor(10);
    printf("\n[Done] 数据已自动保存到本地。 \n");
    setColor(7);
}
```

// 加载数据

```
void loadFromFile() {
```

// 二进制只读模式("rb") 打开磁盘上的数据文件。

```
FILE* fp = fopen("flights.dat", "rb");
```

```
if (fp == NULL) return;
```

//在栈(Stack)上开辟的临时结构体变量, 负责承接从硬盘里读取的数据。

```
Flight temp;
```

// fread 按照 sizeof(Flight), 从文件中每次读取一个完整的结构体块数据, 并存入私有的 temp 空间。

// 若读取成功, 返回值为 1, 循环继续; 直到读完文件末尾 EOF, 循环停止。

```
while (fread(&temp, sizeof(Flight), 1, fp)) {
```

// 每读到一个航班, 在堆(Heap)上通过 malloc 重新申请一块永久的内存。

// 通过结构体直接赋值(浅拷贝), 把 temp 里的数据克隆到新内存中。

```
FlightPtr newF = (FlightPtr)malloc(sizeof(Flight));
```

```
*newF = temp;
```

// 保存在磁盘二进制文件里的 bookedList 和 waitList 实际上是上一次运行时的内存地址(指针)。

// 当程序重启后, 这些地址已经失效, 变成“野指针”。

// 因此, 需要重置内存的指针。

```
newF->bookedList = NULL;
```

```
newF->waitList = NULL;
```

```
newF->next = flightList;
```

```
flightList = newF;
```

```
}
```

```
fclose(fp);
```

```
}
```

// 主函数

```
int main() {
```

```
    initConsole();
```

```
    loadFromFile(); // 启动时加载已保存数据
```

```
    int choice = -1;
```

